MySQL : Bien choisir le type de données

1. Pourquoi le choix du type de données est crucial ?
Le choix du type de données dans MySQL est une **décision fondamentale** lors de la création d'une table.
Il permet de :

- **Optimiser l'espace disque** et la **vitesse des requêtes**
- **Garantir la précision des calculs** (notamment en comptabilité)
- **Assurer l'intégrité des données** (éviter les valeurs incorrectes)
- **Faciliter les recherches, index et jointures**

Un mauvais type de données entraîne :

- Sur-stockage inutile
- Ralentissement du système
- Erreurs d'arrondi ou incohérences

2. Catégorie « Caractère »
2.1 CHAR(n) – longueur fixe

- Stocke toujours **n caractères**
- Rapide pour les valeurs de taille constante

Exemples d'utilisation

- Codes pays (ISO)
- Codes employés

code_pays CHAR(2)
Exemples : TG, FR, US

2.2 VARCHAR(n) – longueur variable

- Stocke uniquement la longueur réelle
- Économise de l'espace

Exemples d'utilisation

- Noms
- Emails
- Adresses

email VARCHAR(100)

2.3 TEXT et BLOB – grands contenus

- **TEXT** : texte long (commentaires, articles)
- **BLOB** : données binaires (images, PDF)

Exemples
description TEXT
photo BLOB
⚠️ À éviter dans les clauses WHERE et les clés primaires.

2.4 ENUM et SET – liste fermée

- **ENUM** : une seule valeur autorisée
- **SET** : plusieurs valeurs possibles

Exemples

statut ENUM('nouveau','en_cours','livre')
tags SET('urgent','vip','retard')

3. Catégorie « Numérique »

3.1 Entiers

Type	Taille	Exemple
TINYINT	1 octet	âge
INT	4 octets	identifiant
BIGINT	8 octets	transactions bancaires

age TINYINT UNSIGNED

3.2 DECIMAL(p,s) – valeurs exactes

- Utilisé pour l'**argent**
- Aucun arrondi imprévu

salaire DECIMAL(10,2)
Exemple : 125000.50

3.3 FLOAT / DOUBLE – valeurs approximatives

- Calculs scientifiques
- Capteurs, statistiques

temperature FLOAT
⚠️ À éviter pour la comptabilité.

3.4 Booléen

MySQL ne possède pas de vrai BOOLEAN.
actif BOOL
➞ équivaut à TINYINT(1)
1 = vrai, 0 = faux

4. Catégorie « Date / Heure »

4.1 DATE

date_naissance DATE
Exemple : 2001-08-15

4.2 TIME

heure_debut TIME
Exemple : 08:30:00

4.3 DATETIME

- Date + heure complètes
- Jusqu'à l'an 9999

date_commande DATETIME

4.4 TIMESTAMP

- Basé sur l'heure UTC
- Mise à jour automatique possible

created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP

4.5 YEAR
annee YEAR
Exemple : 2026

5. Mini-checklist pour créer une table
Avant de créer une table, toujours se poser ces questions :

- Quel est **le type le plus petit possible** couvrant les valeurs ?
- Les valeurs peuvent-elles être **négatives** ?
  - Sinon → UNSIGNED
- Cette colonne sera-t-elle :
  - recherchée ?
  - utilisée en jointure ?
  - indexée ?
- Éviter TEXT et BLOB comme clé ou condition

6. Exemple simple et complet

```
CREATE TABLE orders (
   order_id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
   customer_name VARCHAR(50) NOT NULL,
   order_date DATETIME NOT NULL,
   total_price DECIMAL(10,2) NOT NULL,
   status ENUM('new','ship','deliv') DEFAULT 'new'
);
```

Pourquoi ces choix ?

- INT UNSIGNED : pas de valeurs négatives
- VARCHAR(50) : noms variables
- DATETIME : historique précis
- DECIMAL(10,2) : montant exact
- ENUM : statut contrôlé

7. À retenir (message clé aux étudiants)
Bien typer ses données, c'est construire une base :

- plus légère
- plus rapide
- plus fiable

Un mauvais choix de type peut provoquer des **erreurs graves**, surtout en **gestion et comptabilité**.